

Exercises for Week 8

1. Dataframe Chunking

Sometimes, you need to analyze tabular data in a CSV file which is too large to load into memory. In this exercise, we'll build on the last exercise in the previous week and adapt it for a memory constrained environment. Specifically, we will use the chunking functionality from Pandas and PyArrow to process a dataframe in small bits without losing significant performance.

Briefly, the last exercise involved taking a dataframe with weather data from DMI (Danmarks Meteorologiske Institut) and computing the total amount of precipitation across all measurement stations. You may find an example solution to this in [last week's solutions file on Learn](#).

The dataframe is at: `/dtu/projects/02613_2025/data/dmi/2023_01.csv.zip`

1. **Autolab** Make a Python program which takes two command line arguments: a path to a Pandas data frame and a chunk size. Compute the total precipitation where you process the dataframe in chunks of the given size - do *not* load the entire dataframe. Also, do *not* perform any memory reducing operations (like in week 7). Finally, print the result (and only the result).
2. Measure the run time and memory use for chunk sizes 1,000, 10,000, 100,000 and 1,000,000. You can measure the run time and memory use using the `/usr/bin/time` command as

```
1 /usr/bin/time -f"mem=%M KB runtime=%e s" python script.py
```

Perform the measurement as a batch job. Select 1 core and a specific CPU so the results are repeatable. What do you observe? How does the run time compare to the non-chunked version? Hint: **time prints its output to stderr** and will be added to the '.err' file. If you want, you can redirect it to stdout by adding `2>&1` at the end of your timing command, e.g, `/usr/bin/time <stuff> 2>&1`.

3. Convert the CSV file to a chunked parquet file. Use a chunksize of 100,000. Hint: See section 8.3.2 in Fast Python.
4. Modify your Python program from the previous exercise to receive the path to a chunked parquet file instead of a path to a CSV file. What is the new run time and memory use? As before, perform your measurement with a batch script and make sure you select the *same* CPU model.
5. Finally, modify your program further to only load the relevant columns. What is your final run time?

2. Mandelbrot memmap

In this exercise, we expand on the exercise from weeks 5 and 6 where we generated the Mandelbrot set by modifying the code to work in a memory constrained environment. We will now store the data in a memory mapped array, which would allow us to generate results larger than memory. For these exercises however, we will stick to sizes which can still fit in order to make the run times manageable.

1. **Autolab** Make a Python program that creates an $N \times N$ array with the Mandelbrot set. It must take the number N as a command line argument. The limits for the real and imaginary values must be the same as in weeks 5 & 6. The program must store the data for the Mandelbrot array in a memory mapped array.
2. Parallelize your program for computing the Mandelbrot set in a memory mapped array by modifying your solution from weeks 5 and 6. Each process should write its part of the Mandelbrot array directly to the memory mapped array.
 - a) Run the program for a 1000 x 1000 array. What is your speed-up? What is your run time compared to the in-memory version from weeks 5 and 6. Run your measurement as a batch job. Select a specific CPU model so your results are repeatable.
3. **Autolab** Make a Python program which reads the saved Mandelbrot array, downsamples it, and saves the result as a PNG image. It must take three command line arguments: the path to the saved Mandelbrot array from exercise 2.1, the size N of the $N \times N$ array and an integer step length n . It must then open the specified array as a NumPy memmap, load every n th row and column, and then save the resulting array as a PNG file. Hint: see the example code from exercise 3 in week 5.

Example: Assume the path is to the following 5×5 array:

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix}$$

Given a step length of $n = 3$, the downsampled array is

$$\begin{bmatrix} 1 & 4 \\ 16 & 19 \end{bmatrix}$$

Given a step length of $n = 2$, the downsampled array is

$$\begin{bmatrix} 1 & 3 & 5 \\ 11 & 13 & 15 \\ 21 & 23 & 25 \end{bmatrix}$$

4. At `/dtu/projects/02613_2025/data/mandelbrot/mandelbrot.raw` is a file with a 4000×4000 Mandelbrot set array. Run the downsampling program on this array for the step lengths 1, 2, 4, 8, 16. If you hardcoded the size of the memory mapped array 1000×1000 , remember to change it.

Measure the run time and peak memory use with the `/usr/bin/time` command. Plot the results. Perform the measurements as a batch job and select a CPU model so the results are repeatable. What happens to the memory and run time as the step length n increases?

3. Mandelbrot Zarr

1. **Autolab** Make a Python program that creates a Zarr array containing the Mandelbrot set. The size of the array must be $N \times N$. The program must take two command line arguments. The size of the array N and a positive integer C which is the size of the $C \times C$ chunks. Structure your program so it fills out the chunks one at a time.
2. Parallelize your program using multiprocessing. Each process should fill one chunk of the Zarr array.
3. Time the runtime of the program for array size 1000 and chunk sizes 10, 25, 50, 100 and 200. Your program should make a process for each available core, e.g., for a processor with 32 cores use 32 processes. Run your timing as a batch job where you specify a CPU model so your result is repeatable.
 - a) What happens with the run times? What chunk size gives the fastest execution?
 - b) What happens with the size of the saved Zarr array? What chunk size gives the smallest result? Hint: since the Zarr array is saved in a directory you can measure its size with the command `du -sh <filename>.zarr`.
4. How does the performance compare with your memmap implementation?